

OO2 — Patrones de Diseño

Cheatsheet · UNLP 2026 · Adapter · Decorator · Composite · Template Method · Strategy · State · Observer · Null Object · Frameworks

ESTRUCTURALES

Adapter

¿QUÉ HACE?

Convierte la interfaz de una clase en otra que el cliente espera. Hace compatibles clases con interfaces incompatibles.

¿CUÁNDO?

Querés usar una clase existente pero su interfaz no encaja con lo que necesitás (ej: librería externa no modificable).

ROLES

Target · Client · Adaptee · Adapter

CLAVE

Adapter extends Target, tiene composición con Adaptee.

EJEMPLO

```
TelegramAdapter extends Actuador {
  adaptado.notify(msg); }
```

Decorator

¿QUÉ HACE?

Agrega responsabilidades a un objeto dinámicamente, sin modificar su clase ni usar herencia masiva.

¿CUÁNDO?

Necesitás combinar libremente N servicios/funciones sobre un objeto base (ej: servicios de envío combinables).

ROLES

Component · ConcreteComponent · Decorator · ConcreteDecorator

CLAVE

Decorator implementa Component y tiene un Component (wrapped).

EJEMPLO

```
new Express(new
ConSeguro(paquete))
.getCostoEnvio()
```

Composite

¿QUÉ HACE?

Compone objetos en estructuras de árbol. El cliente trata a objetos simples y compuestos de la misma forma.

¿CUÁNDO?

Tenés jerarquías parte-todo (ej: archivos y carpetas, menús y submenús).

ROLES

Component · Leaf · Composite

CLAVE

Composite implementa Component y tiene lista de Component hijos.

EJEMPLO

```
carpeta.add(archivo);
carpeta.getSize(); // suma
recursiva
```

DE COMPORTAMIENTO

Template Method

¿QUÉ HACE?

Define el esqueleto de un algoritmo en la superclase, dejando que las subclases implementen pasos específicos sin cambiar la estructura.

¿CUÁNDO?

Varias clases hacen los mismos pasos en el mismo orden, pero con distinta implementación en algunos. Evita duplicación.

ROLES

AbstractClass (templateMethod + primitive ops) · ConcreteClass

CLAVE

templateMethod() es final o no override. Primitive ops son abstract. Inversión de control: la superclase llama a las subclases.

EJEMPLO

```
abstract class Actuador {
  void update(Sensor s) {
    registrarCambio(s);
    actuarAnteCambio(s); // hook
  }
}
```

Strategy

¿QUÉ HACE?

Define una familia de algoritmos, encapsula cada uno y los hace intercambiables. El cliente elige el algoritmo en runtime.

¿CUÁNDO?

Tenés múltiples variantes de un comportamiento y querés poder cambiarlas dinámicamente (vs Template Method que usa herencia).

ROLES

Context · Strategy (interfaz) · ConcreteStrategy

CLAVE

Context tiene una Strategy por composición. Diferencia con Template Method: composición vs herencia.

EJEMPLO

```
new ReportGenerator(new
PDFStrategy())
.generateReport(doc);
```

State

¿QUÉ HACE?

Permite que un objeto cambie su comportamiento cuando cambia su estado interno. El objeto parece cambiar de clase.

¿CUÁNDO?

Un objeto tiene comportamiento que depende de su estado y debe cambiar en runtime según ese estado (ej: semáforo, pedido).

ROLES

Context · State (interfaz) · ConcreteState

CLAVE

Context delega en el State actual. El State puede cambiar el estado del Context.

EJEMPLO

```
pedido.pagar(); // delega en
estado
// PedidoPendiente →
PedidoPagado
```

Observer

¿QUÉ HACE?

Define dependencia uno-a-muchos: cuando un objeto cambia de estado, todos sus dependientes son notificados automáticamente.

¿CUÁNDO?

Cambios en un objeto deben propagarse a otros sin acoplarlos (ej: sensores e IoT, eventos UI, MVC).

ROLES

Subject · Observer (interfaz) · ConcreteObserver

CLAVE

Subject tiene lista de Observer. Llama update() en cada uno. Observers se suscriben/desuscriben.

EJEMPLO

```
sensor.agregarSuscriptor(ventilador);
sensor.setValor(20); // notifica a todos
```

Null Object

¿QUÉ HACE?

Provee un objeto con comportamiento nulo que implementa la misma interfaz, evitando chequeos de null.

¿CUÁNDO?

Tenés muchos if (obj != null) desperdigados. El caso 'sin objeto' tiene un comportamiento bien definido (no hacer nada).

ROLES

AbstractObject · RealObject · NullObject

CLAVE

NullObject implementa la interfaz con métodos vacíos o sin efecto. Cliente nunca chequea null.

EJEMPLO

```
// sin null object:
if (logger != null) logger.log();
// con null object:
logger.log(); // NullLogger no hace nada
```

CREACIONALES / FRAMEWORK

Frozen/Hot Spots

¿QUÉ HACE?

Conceptos de framework: Frozen spot = comportamiento fijo (no extension point). Hot spot = punto de extensión que el usuario implementa.

¿CUÁNDO?

Diseñando o usando un framework. El framework define el flujo (frozen), vos implementás los hotspots.

ROLES

Frozen spot: lógica invariante del framework. Hot spot: operaciones abstractas que extiende el usuario.

CLAVE

Template Method implementa hotspots via herencia. Strategy los implementa via composición (más flexible).

EJEMPLO

```
// Robot Wars: step() es frozen
// avanzar(), disparar() son hot spots
// Vos implementás tu robot extendiendo
```

Inversión de Control

¿QUÉ HACE?

El framework llama a tu código, no al revés ('Don't call us, we'll call you'). Característica principal de un framework.

¿CUÁNDO?

Siempre que usás Template Method o Strategy en un framework: el flujo lo controla el framework, no tu aplicación.

ROLES

No es un patrón GoF, es una característica arquitectural.

CLAVE

La superclase llama a métodos definidos en las subclases. Es consecuencia de Template Method.

EJEMPLO

```
// Superclase llama al hook:
void templateMethod() {
  primitiveOp1(); // implementada en subclase
  primitiveOp2();
}
```

TABLA COMPARATIVA DE PATRONES

Patrón	Propósito	Mecanismo	Variabilidad
Template Method	Esqueleto de algoritmo fijo, pasos variables	Herencia	Tiempo de compilación
Strategy	Algoritmo intercambiable en runtime	Composición	Tiempo de ejecución
State	Comportamiento según estado interno	Composición	Tiempo de ejecución
Adapter	Compatibilizar interfaces incompatibles	Composición + Herencia	Tiempo de compilación

Decorator	Agregar responsabilidades dinámicamente	Composición recursiva	Tiempo de ejecución
Observer	Notificar cambios a múltiples dependientes	Composición (lista)	Tiempo de ejecución

CODE SMELLS Y REFACTORINGS

Code Smell	Refactoring	Qué hace
Long Method	Extract Method	Extrae fragmento a método con nombre expresivo
Duplicated Code (subclases)	Pull Up Method	Sube método idéntico a la superclase
Duplicated Code (algoritmo)	Form Template Method	Sube el esqueleto, deja los pasos como abstract
Switch Statement	Replace Conditional with Polymorphism	Cada rama del if/switch pasa a ser una subclase
Comments	Remove Comment / Rename	El nombre del método reemplaza el comentario
Feature Envy	Move Method	El método pasa a la clase de los datos que usa
Loop innecesario	Replace Loop with Pipeline	Usa <code>stream().filter().map().collect()</code>